

System Software Specification

Custom Heap Memory Allocator

Prepared for: Software Engineering Faculty

Date: November 4, 2025

1. Introduction

1.1 Purpose

The Software Requirement Specification (SRS) defines the complete set of functional and non-functional requirements for the Custom Memory Allocator, and educational memory management library written in C.

The allocator enables students and practitioners of systems programming to understand dynamic memory management mechanisms through hands-on implementation and deployment of segregated free list architectures, allocation strategies, thread synchronization primitives, and heap management protocols.

1.2 Document Conventions

This document adheres IEEE Standard and employs the following

- **Shall:** Mandatory requirements that must be implemented
- **Should:** Recommended requirements that enhance quality but are not strictly mandatory
- **May:** Optional features that may be implemented
- **High/ Medium/Low Priority:** Indicates relative importance for development and effort allocation.

1.3 Product Scope

The Custom Heap Memory Allocator is a user-space C library providing dynamic memory allocation services analogous to standard `malloc()`, `free()`, `calloc()`, and `realloc()`, but implemented from first principles for educational impact. It does not replace system allocators but serves as a pedagogical tool demonstrating core allocation mechanism.

1.4 References

1. IEEE Computer Society. IEEE Recommended Practice for Software Requirements Specification. Institute of Electrical and Electronics Engineers, 1998.
 2. Custom Heap Memory Allocator GitHub Repository (Internal).
-

2. Overall Description

2.1 Product Perspective

The Custom Heap Memory Allocator operates as a self-contained library component between application code and the operating system's virtual memory subsystem, acquiring heap space via POSIX `sbrk()` calls and managing that space through structured metadata and free list organization.

2.2 Product Features

The allocator provides four core memory management operations with supporting infrastructure:

- **Memory Allocation:** Requesting heap blocks of specified size with alignment guarantees
- **Memory Deallocation:** Returning blocks to the free pool with automatic coalescing
- **Zero-Initialized Allocation:** Array allocation with automatic zeroing
- **Segregated Free List Management:** Organization into 8 size classes for efficient searching
- **Block Splitting:** Dividing oversized blocks to minimize intern fragmentation.
- **Block Coalescing:** Merging adjacent free blocks to minimize external fragmentation.
- **Thread-safe Heap Extension:** Mutex-protected `sbrk()` calls for concurrent safety
- **Memory Alignment:** Enforcing 8-byte (configurable) alignment boundaries

2.3 Operating Environment

Hardware: x86-64 or ARM64 processor; 4 KB minimum available heap; 8-byte alignment support.

Software: Linux kernel 3.10+; GCC 7.0+ or Clang 6.0+; POSIX thread support; standard C libraries.

Runtime: Single or multi-threaded POSIX applications; user-space execution; standard heap availability.

2.4 Design and Implementation Constraints

- No use of `malloc()`, `free()`, `calloc()`, or `realloc()` internally
- Metadata stored immediately before user data
- Thread synchronization via POSIX `pthread_mutex_t` only
- Heap growth only through `sbrk()` system calls

2.6 Assumptions and Dependencies

- Operating system provides functional `sbrk()` implementation
- Application code correctly pairs allocations with deallocations
- Multi-threaded applications properly initialize POSIX threads

- Compiler respects memory alignment requirements
 - No garbage collection or automatic memory recovery
-

3. System Requirements

3.1 System-level Overview

The Custom Heap Memory Allocator shall function as a self-contained memory management subsystem that:

- Maintains complete isolation from standard system allocators
- Manage a private heap pool obtained exclusively through `sbrk()` system calls
- Supports both single-threaded and multi-threaded application contexts

3.2 System Interfaces

SR-1: Kernel Interfaces

The allocator shall interact with that operating system kernel exclusively through the POSIX `sbrk()` system call for heap extension. All `sbrk()` calls shall be protected by global mutex to ensure atomicity in multi-threaded contexts.

SR-2: Thread Library Integration

The allocator shall initialize and utilize POSIX `thread_mutex_t` mutexes for synchronization. The allocator shall be compatible with applications using POSIX threads (`pthread_create()`, `pthread_join()`, etc.).

SR-3: Process Heap Management

The allocator shall operate within the process's standard heap region, not allocating memory-mapped regions or shared memory segments.

SR-4 System Dependencies;

The system depends upon:

- POSIX-compliant operating system (Linux, macOS)
 - Standard C runtime library (libc)
 - POSIX thread library (libpthread)
 - Functional `sbrk()` system call implementation
-

4. External Interface Requirements

4.1 Application Programming Interface (API)

The allocator exposes four primary functions and optional inspection functions:

Primary API functions:

// Allocate uninitialized memory

void* my_alloc(size_t size);

// Deallocate previously allocated memory

void freee(**void*** ptr);

// Allocate zero-initialized memory (array-like)

void* call_oc(size_t nelem, size_t elsize);

// Resize previously allocated memory

void* reall_oc(**void*** ptr, size_t size);

Optional Inspection Functions:

// Retrieve total heap space acquired from OS

size_t get_heap_size(**void**);

// Retrieve total bytes currently in free lists

size_t get_free_bytes(**void**);

// Output diagnostic statistics (debug builds)

void print_heap_stats(**void**);

4.2 Function Contracts

FR-API-1.1: my_alloc() Contract

Aspect	Specification
Precondition	Allocator initialized (implicit on first call); size > 0
Input	size (type: size_t): Bytes requested, range 1 to SIZE_MAX - metadata overhead
Processing	Search segregated free lists; split oversized blocks; extend heap if necessary; format metadata

Aspect	Specification
Output	Pointer to aligned, usable memory or NULL
Postcondition	Memory allocated and metadata correctly initialized; suitable free block added to list if split occurred
Error Cases	size == 0 → return NULL; allocation overflow → return NULL; sbrk() failure → return NULL

FR-API-1.2: free() Contract

Aspect	Specification
Precondition	ptr == NULL or valid allocator-returned pointer; no prior free on this pointer
Input	ptr (type: void*): Pointer to deallocate
Processing	Validate pointer; mark block free; check and coalesce adjacent blocks; insert into appropriate free list
Output	None (void return)
Postcondition	Block marked free; coalescing performed; free list updated
Error Cases	Invalid pointer → undefined behavior (assertion in debug); double-free → assertion failure

FR-API-1.3: call_oc() Contract

Aspect	Specification
Precondition	nelem > 0, elsize > 0; product does not overflow size_t
Input	nelem (type: size_t): Element count; elsize (type: size_t): Bytes per element
Processing	Compute total_size = nelem * elsize with overflow detection; call my_alloc(total_size); zero all bytes
Output	Pointer to zero-initialized memory or NULL
Postcondition	Memory allocated; all bytes set to zero; metadata reflects actual allocation size
Error Cases	nelem * elsize overflows → return NULL; either parameter zero → return NULL; allocation failure → return NULL

FR-API-1.4: reall_oc() Contract

Aspect	Specification
Precondition	ptr == NULL or valid allocator-returned pointer; original block not freed
Input	ptr (type: void*): Existing allocation; size (type: size_t): New desired size
Processing	If ptr == NULL: equivalent to my_alloc(size); if size == 0: equivalent to free(ptr) returning NULL; otherwise: attempt in-place expansion, or allocate new block, copy data, free old block
Output	Pointer to resized block or NULL on failure
Postcondition	Data preserved up to min(old_size, new_size); old block freed if relocation occurred
Error Cases	Reallocation failure → return NULL, original block remains valid

4.3 Data Structure Interfaces

Metadata Block Structure (Internal, not exposes to API):

```
struct meta_block {
    size_t actual_size;    // Total block size including metadata header
    size_t requested_size; // Original user request size
    int is_free;           // 0 = allocated, 1 = free
    struct meta_block* next; // Free list linkage pointer
};
```

Structure Invariants:

- actual_size >= requested_size + sizeof(struct meta_block)
- actual_size is a multiple of alignment boundary (8 bytes)
- is_free is binary (0 or 1 only)
- next is valid pointer or NULL

4.4 Error Reporting Mechanism

Errors are reported through return values:

- Allocation functions (my_alloc, call_oc, reall_oc) return NULL on failure
- DFEallocation (free) returns void; errors trigger assertions in debug builds
- System integration errors (e.g. sbrk()) failure) propagate as allocation failures

5. Functional Requirements

5.1 Memory Allocation Operations

FR-5.1.1: Block Allocation with First-fit Strategy [High Priority]

The allocator shall implement a **first-fit allocation strategy**. When an allocation request arrives:

1. Determine the size class corresponding to the rounded-up request size
2. Search the free list for that size class from the head
3. Return the first free block that satisfies `block.actual_Size >= requested_size`
4. If no block is found in the current class, search progressively larger size classes
5. If no suitable free block exists, extend the heap via `sbrk()` and allocate from new space

Rationale: First-fit provides $O(1)$ best-case allocation time and demonstrates fundamental allocation strategy trade-offs.

FR-5.1.2: Block Splitting on Oversized Free Blocks [High Priority]

When a free block larger than the requested size is selected for allocation:

1. Calculate remainder size: `remainder = block.actual_size - requested_size - sizeof(struct meta_block)`
2. If `remainder >= 32` bytes (minimum useful split threshold):
 - 2.2 Allocate requested portion with new metadata
 - 2.3 Create new free block from remainder with correct metadata
 - 2.4 Insert remainder block into appropriate size class list
3. If `remainder < 32` bytes:
 - Allocate entire block (accept internal fragmentation)

Rationale: Splitting reduces internal fragmentation; minimum threshold prevents creation of unusable fragments.

FR-5.1.3: Memory Alignment [High Priority]

All allocated memory blocks shall satisfy alignment requirements:

1. Default alignment boundary shall be 8 bytes (configurable via compile-time constant)
2. All returned pointers shall be multiple of the alignment boundary

3. Requested sizes shall be rounded up to nearest multiple of alignment before allocation
4. Metadata storage shall account for alignment, ensuring user data begins at aligned address

Rationale: Alignment ensures efficient CPU access and compliance with architecture requirements.

FR-5.1.14: Metadata Initialization [High Priority]

Upon allocating a block, the allocator shall initialize metadata fields:

1. `actual_size`: Set to the actual allocated size (including metadata header)
2. `requested_size`: Set to the user-requested size (before rounding)
3. `is_free`: Set to 0 (allocated)
4. `next`: Set to NULL (not in free list)

5.2 Memory Deallocation Operations

FR-5.2.1: Block Deallocation and List Insertion [High Priority]

When `free(ptr)` is invoked:

1. Validate that `ptr` is not NULL (if NULL, silently return per POSIX semantics)
2. Retrieve metadata by pointer arithmetic: `meta = (struct meta_block*)ptr - 1`
3. Validate metadata integrity (detect obvious corruption)
4. Check if block is already free (double-free detection); assert or handle gracefully
5. Mark block as free: `meta->is_free = 1`
6. Perform coalescing (see FR-5.2.2)
7. Insert resulting free block into appropriate size class list at head (O(1) insertion)

FR-5.2.2: Adjacent Block Coalescing [High Priority]

After marking a block as free, the allocator shall:

Forward Coalescing:

1. Calculate the address of the next block: `next_block = (void*)ptr + actual_size`
2. Check if `next_block` is within allocated heap space

3. If yes, retrieve its metadata and check `is_free == 1`
4. If free, merge: update current block's `actual_size` to include next block;
remove next block from free list

Backward Coalescing:

1. Iterate through allocator's tracking structures to find the block immediately preceding current block in memory
2. Check if predecessor is free
3. If yes, merge: update predecessor's `actual_size` to include current block;
remove current block metadata

Postcondition: All adjacent free blocks merged into single contiguous free block with unified metadata; result inserted into appropriate size class list.

FR-5.2.3: Segregated Free List Management [High Priority]

The allocator shall maintain 8 segregated free lists organized by size classes:

Class Min (bytes) Max (bytes)

0	1	32
1	33	64
2	65	128
3	129	256
4	257	512
5	513	1024
6	1025	2048
7	2049+	∞

List Operations:

- **Insert at Head:** $O(1)$ insertion when block becomes free
- **Search from Head:** $O(n)$ in worst case where n is blocks in size class
- **Cascading Search:** If no block in size class k , search class $k+1$, $k+2$, etc.

5.3 Special Allocation Operations

FR-5.3.1: Zero-Initialized Allocation (call_oc) [High Priority]

The call_oc(nelem, elsize) function shall:

1. Detect overflow: if nelem * elsize overflows size_t, return NULL
2. Call my_alloc(nelem * elsize) to allocate memory
3. If allocation succeeds, zero all bytes: memset(ptr, 0, nelem * elsize)
4. Return pointer to zero-initialized memory

Postcondition: Returned memory initialized to all zeros; metadata reflects actual allocation.

FR-5.3.2: Block Reallocation (reall_oc) [High Priority]

The reall_oc(ptr, size) function shall:

Case 1: ptr == NULL

- Equivalent to my_alloc(size)

Case 2: size == 0

- Equivalent to free(ptr), return NULL

Case 3: size < old_size

- Check if block can be split
- If remainder after split is ≥ 32 bytes, perform split and free remainder
- Otherwise, keep entire block allocated
- Return original pointer

Case 4: size > old_size

- Check if in-place expansion is possible (next block free and large enough)
- If yes: merge with next block, update metadata, return original pointer
- If no: allocate new block, copy old_size bytes, call free(ptr), return new pointer

Postcondition: Data preserved up to min(old_size, new_size); original block invalid after relocation.

5.4 Thread Safety Requirements

FR-5.4.1: Mutex-Protected Heap Extension [High Priority]

All calls to `sbrk()` shall be protected by a global mutex:

c

```
static pthread_mutex_t heap_mutex = PTHREAD_MUTEX_INITIALIZER;
```

```
void* safe_sbrk(intptr_t increment) {  
    pthread_mutex_lock(&heap_mutex);  
    void* result = sbrk(increment);  
    pthread_mutex_unlock(&heap_mutex);  
    return result;  
}
```

Rationale: `sbrk()` is not thread-safe; mutual exclusion ensures only one thread extends heap at a time.

FR-5.4.2: Non-Blocking Free List Operations [Medium Priority]

Free list manipulations within existing heap space (insertions, removals) do not require mutex protection under the assumption that the allocator operates on a single heap pool per process.

5.5 Error Handling Requirements

FR-5.5.1: Zero-Size Allocation Rejection [High Priority]

`my_alloc(0)` shall return NULL without modifying allocator state.

FR-5.5.2: Integer Overflow Detection [High Priority]

`call_oc(nelem, elsize)` shall detect if `nelem * elsize` overflows `size_t`:

c

```
if (elsize != 0 && nelem > SIZE_MAX / elsize) {  
    return NULL; // Overflow detected  
}
```

FR-5.5.3: Double-Free Detection [High Priority]

When `free(ptr)` is called:

1. Retrieve metadata
2. Check if `is_free == 1` (block already freed)
3. In debug builds: trigger assertion failure
4. In release builds: implementation may attempt recovery or graceful exit

FR-5.5.4: Invalid Pointer Handling [High Priority]

If `free(ptr)` is called with a pointer not returned by the allocator:

1. Metadata retrieval may fail or retrieve corrupted data
2. In debug builds: assertion failure or detection error
3. In release builds: undefined behavior (acceptable per specification)

FR-5.5.5: Heap Exhaustion Handling [High Priority]

If `sbrk()` returns `(void*) -1` (indicating failure):

1. The pending allocation request shall return NULL
2. The allocator shall remain in consistent state
3. Future allocation attempts may succeed if application releases memory

FR-5.5.6: NULL Pointer Tolerance [High Priority]

- `free(NULL)` shall silently return without error
 - `realloc(NULL, size)` shall behave as `my_alloc(size)`
-

6. Non-Functional Requirements

6.1 Performance Requirements

NFR-6.1.1: Allocation Time Complexity [High Priority]

- **Best case:** $O(1)$ when suitable free block is at list head
- **Average case:** $O(k)$ where k is average blocks in searched size classes
- **Worst case:** $O(n)$ where n is total free blocks across all size classes
- **Target:** 100,000+ allocations per second for 64-byte blocks on x86-64

NFR-6.1.2: Deallocation Time Complexity [High Priority]

- **Best case:** $O(1)$ for list insertion
- **Coalescing case:** $O(k)$ where $k \leq 2$ (at most two adjacent blocks)
- **Target:** 100,000+ deallocations per second

6.2 Reliability Requirements

NFR-6.2.1: Crash Resistance [High Priority]

The allocator shall not crash or enter inconsistent states during:

- Intensive allocation/deallocation cycles
- Concurrent access from multiple threads
- Edge case handling (exhaustion, invalid pointers)

NFR-6.2.2: Data Integrity [High Priority]

The allocator shall not corrupt user data due to allocator implementation bugs.

NFR-6.2.3: Fragmentation Control [Medium Priority]

External fragmentation shall remain below 30% under typical usage patterns through effective block splitting and coalescing.

6.4 Maintainability Requirements

NFR-6.4.1: Code Documentation [Medium Priority]

All public functions shall include header comments documenting:

- Purpose and semantics
- Parameter descriptions and valid ranges
- Return value specification
- Preconditions and postconditions

NFR-6.4.2: Modular Structure [Medium Priority]

Implementation shall separate concerns into:

- Allocation logic module

- Free list management module
 - Coalescing algorithms module
 - Metadata operations module
-

8. Verification and Validation Criteria

V&V-1: Every functional requirement shall have a corresponding unit test demonstrating correct behavior.

V&V-2: Error handling requirements shall be tested with invalid inputs, overflow conditions, and edge cases.

V&V-3: Thread safety shall be validated through concurrent allocation/deallocation stress tests.

V&V-4: Performance requirements shall be measured on representative target hardware.

9. Appendices

9.1 Glossary

Allocation: The process of assigning a contiguous region of heap memory to an application request.

Block: A contiguous heap region consisting of a metadata header followed by usable data area.

Coalescing: Merging two adjacent free blocks into a single larger free block.

External Fragmentation: Unusable heap space caused by scattered small free blocks.

First-Fit: Allocation strategy selecting the first sufficiently large free block encountered.

Free List: Linked list structure maintaining pointers to free blocks.

Heap: Dynamic memory region managed by the allocator.

Metadata: Administrative information (size, status, linkage) stored with each block.

Segregated Free List: Multiple free lists organized by size ranges.

Splitting: Dividing an oversized free block into allocated and remainder portions.

Software Design Document (SDD)

Custom Heap Memory Allocator

Version: 1.0 Date: November 21, 2025

1. Introduction

1.1 Purpose

The purpose of this Software Design Document (SDD) is to describe the architectural and detailed design of the Custom Heap Memory Allocator. This document translates the requirements defined in the Software Requirement Specification (SRS) into a technical blueprint for implementation. It details the system architecture, data structures, algorithms, and interfaces necessary to build a robust, thread-safe memory management library in C.

1.2 Scope

The Custom Heap Memory Allocator is a user-space library that manages dynamic memory allocation. It interacts with the operating system kernel via the POSIX `sbrk()` system call to acquire heap memory and manages this space using a Segregated Free List mechanism. The design emphasizes educational clarity while maintaining functional equivalence to standard allocators (`malloc`, `free`, etc.).

1.3 Design Goals

- **Modularity:** Separation of allocation logic, heap management, and list manipulation.
- **Safety:** Robust handling of memory boundaries and concurrency via mutex locking.
- **Efficiency:** Utilization of Segregated Free Lists to optimize First-Fit search times (Target: $O(1)$ best case).
- **Compliance:** Strict adherence to 8-byte memory alignment and POSIX thread standards.

2. System Architecture

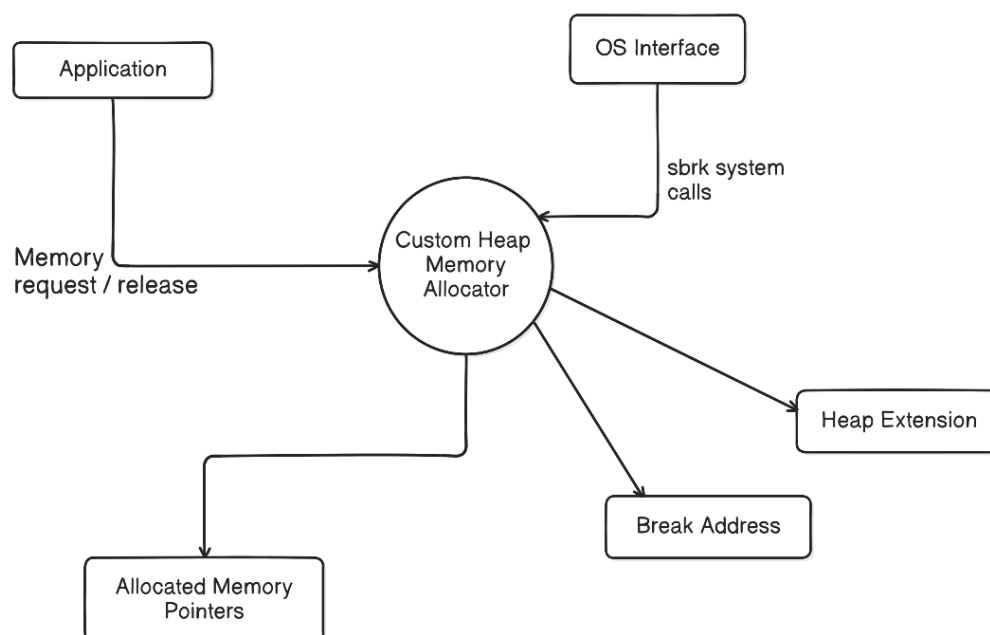
2.1 Architectural Overview

The system follows a **Layered Architecture** pattern. The allocator acts as an intermediary abstraction layer between the Client Application and the Operating System's virtual memory manager.

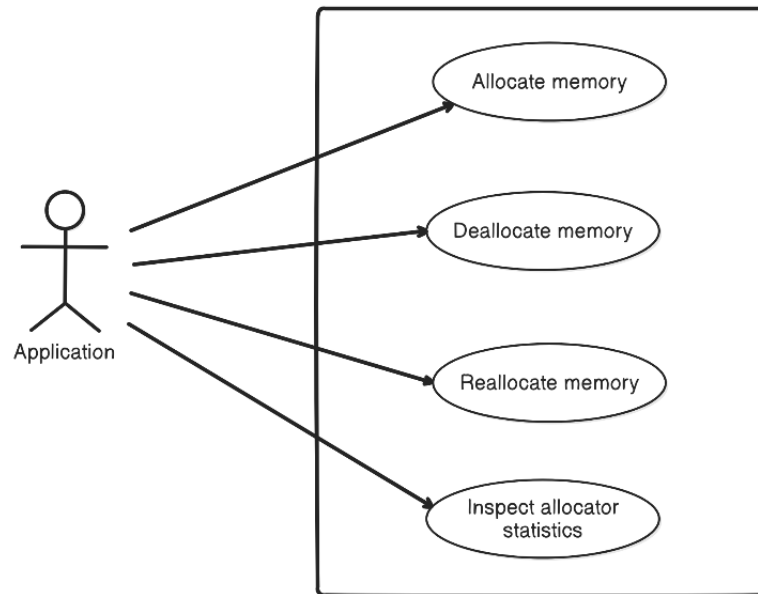
- **Layer 1: Public API Layer:** Exposed interfaces (`my_alloc`, `free`) that validate inputs and route requests.
- **Layer 2: Core Logic Layer:** Implements allocation strategies (splitting, coalescing) and manages free list selection.
- **Layer 3: Heap & Metadata Layer:** Manages the raw heap structure, block metadata formatting, and pointers.
- **Layer 4: System Interface Layer:** Wraps system calls (`sbrk`) with synchronization primitives.

2.2 System Context & Use Cases

- The allocator operates entirely in **User Space**. The Application requests memory, and the Allocator negotiates with the OS Kernel only when the internal pool is exhausted.



(Description: This diagram illustrates the high-level interaction between the Application, the Custom Allocator, and the OS Interface/sbrk calls.)

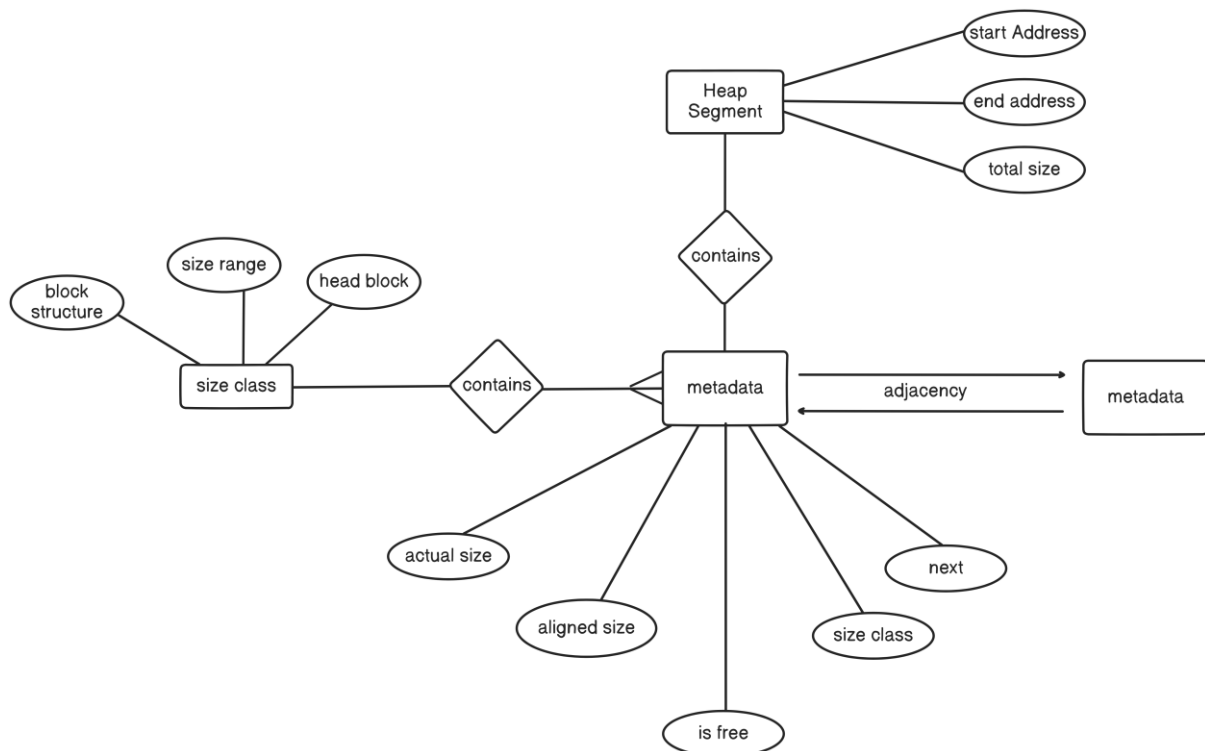


(Description: This diagram details the primary actor (Application) and the supported operations: Allocate, Deallocate, Reallocate, and Inspect Stats.)

3. Data Design

3.1 Internal Data Structures

The allocator relies on a structured metadata system to track memory blocks.



(Description: This diagram visualizes the relationships between Heap Segments, Size Classes, and Metadata attributes like actual_size and is_free.)

3.1.1 Block Metadata (struct meta_block)

Every memory block (allocated or free) is preceded by a metadata header.

```
struct meta_block {  
    size_t actual_size;    // Total block size including metadata (8-byte aligned)  
    size_t requested_size; // Size requested by user (for stats/debugging)  
    int is_free;           // Status flag: 1 = Free, 0 = Allocated  
    struct meta_block* next; // Pointer to next block in the specific Free List  
};
```

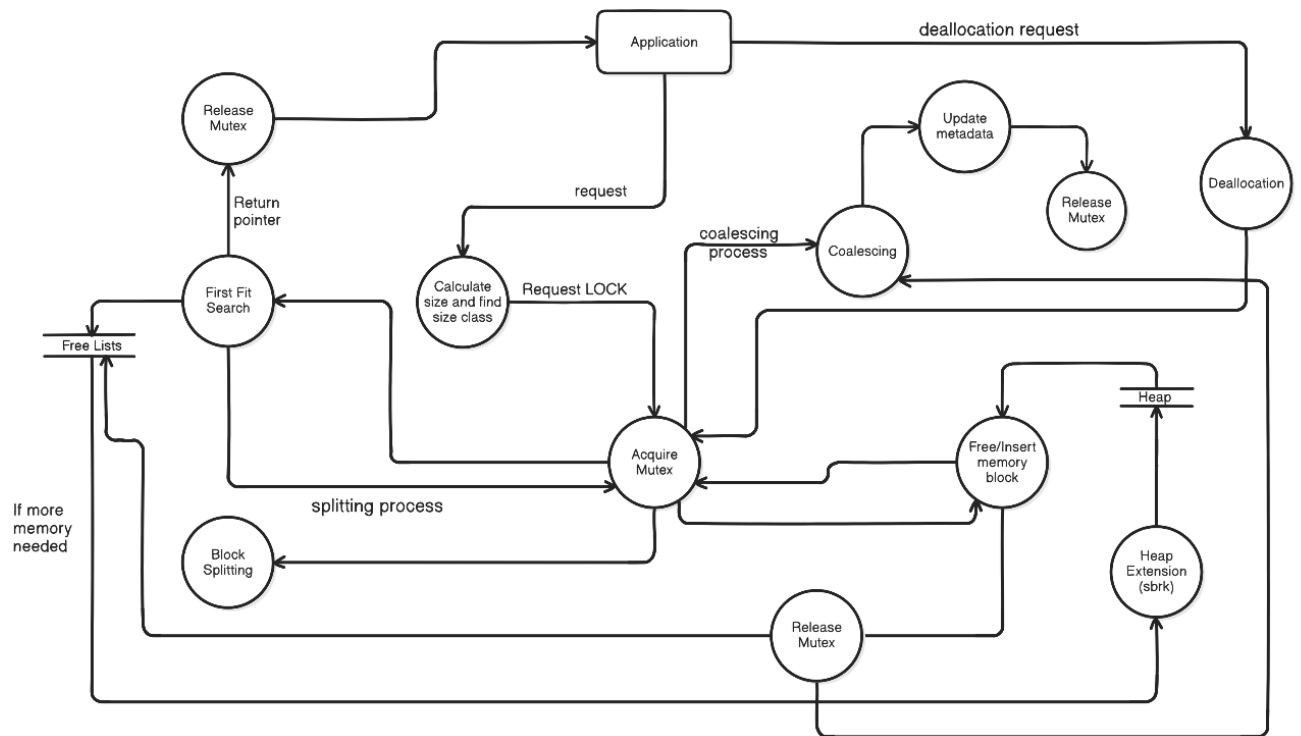
- **Constraint:** sizeof(struct meta_block) must be aligned to 8 bytes.
- **Memory Layout:** [Metadata Header] [User Data Payload ...]

3.1.2 Segregated Free Lists

To optimize search time, the allocator maintains an array of 8 linked lists, categorized by block size. static struct meta_block* free_lists[8];

4. Component Design & Flow

The internal logic requires careful synchronization and state management. The following diagram illustrates the complex flow of control, including locking mechanisms and internal decision loops.



4.1 Module: Allocation Manager

- **Role:** Handles `my_alloc` requests.
- **Logic:**
 1. Aligns requested size.
 2. Selects size class index.
 3. Searches specific `free_list`.
 4. If block found: **Splits** if remainder ≥ 32 bytes.
 5. If not found: Checks larger lists or calls `safe_sbrk`.

4.2 Module: Deallocation & Coalescing

- **Role:** Handles `free` requests.
- **Logic:**
 1. Validates pointer.
 2. Marks `is_free = 1`.
 3. **Forward Coalescing:** Checks `current + size`. If free, merges.
 4. **Backward Coalescing:** Traverses heap to find predecessor. If free, merges.

5. Inserts result into the head of the correct free list.

5. Interface Design

5.1 External API (Public Header)

Function	Signature	Description
Allocation	<code>void * my_alloc(size_t size)</code>	Allocates size bytes. Returns NULL on failure.
Deallocation	<code>void freee(void * ptr)</code>	Frees memory pointed to by ptr.
Zero-Alloc	<code>void * call_oc(size_t nelem, size_t elsize)</code>	Allocates array and initializes to zero. Checks overflow.
Reallocation	<code>void * reall_oc(void * ptr, size_t size)</code>	Resizes existing block. Preserves data up to min size.

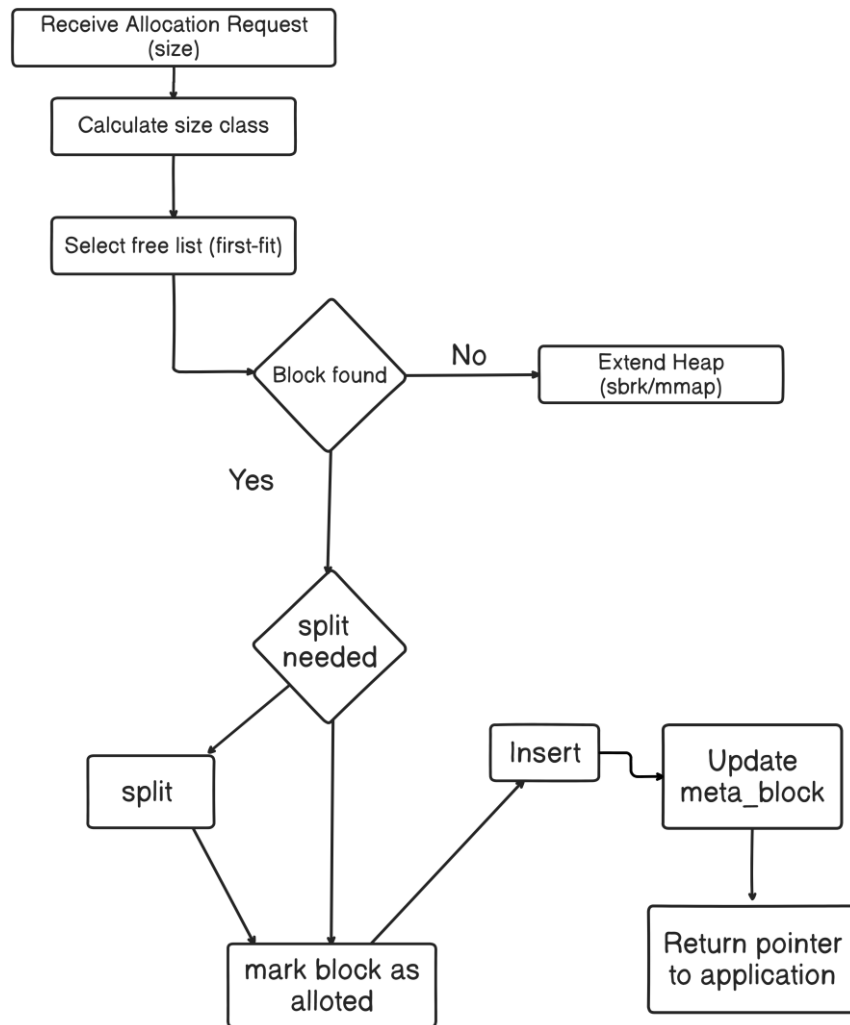
5.2 Internal Interfaces (Private)

- `void* safe_sbrk(intptr_t increment)`: Mutex-protected wrapper for system call.
- `int get_size_class(size_t size)`: Maps size to index 0-7.
- `void list_insert(struct meta_block* b)`: Head insertion ($O(1)$).
- `void split_block(struct meta_block* b, size_t size)`: Splits oversized blocks to reduce internal fragmentation.

6. Dynamic Behaviour

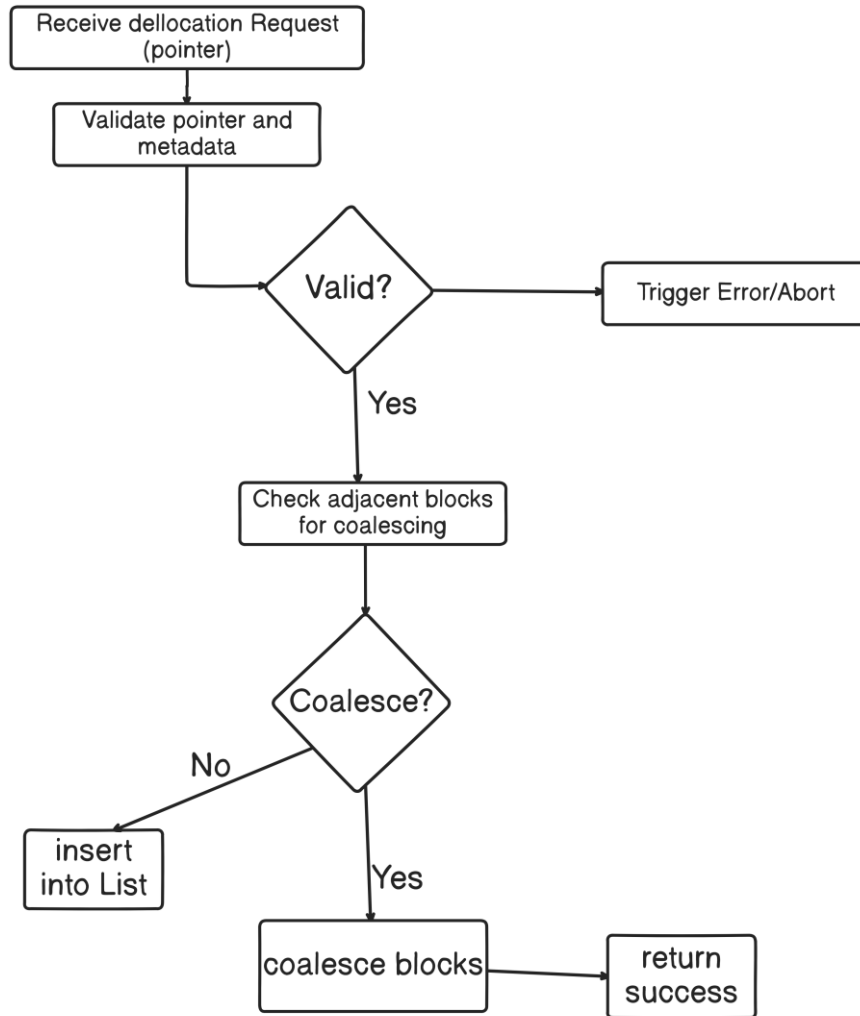
6.1 Allocation Sequence

The following diagram details the decision-making process when a user requests memory.



6.2 Deallocation Sequence

The following diagram details the validation and merging process when a user frees memory.



7. Safety and Reliability

7.1 Thread Safety Strategy

- **Requirement:** SRS FR-5.4.1 (Mutex-Protected Heap Extension).
- **Implementation:**
 - A global `pthread_mutex_t heap_mutex` is initialized at startup.
 - **Critical Section:** The mutex locks **entry** to `my_alloc` and `free` and unlocks on **exit**.
 - This "coarse-grained" locking strategy ensures that no two threads can modify the Free Lists or the Heap Break (`sbrk`) simultaneously, preventing race conditions and heap corruption.

7.2 Error Handling

- **Allocation Failure:** If `sbrk` fails (returns -1), `my_alloc` returns NULL.

- **Double Free:** The system checks if (block->is_free == 1). If true, it aborts (Debug) or returns silently (Release).
- **Integer Overflow:** call_oc validates nelem * elsize before allocation to prevent buffer under-allocation vulnerabilities.

8. Testing Strategy

1. **Unit Testing:** Verify get_size_class maps boundaries correctly (e.g., 32 vs 33 bytes).
2. **Integration Testing:** Run standard workflows (Alloc -> Free -> Alloc) to ensure the free list effectively recycles memory (address reuse).
3. **Stress Testing:** High-frequency threads allocating/freeing to validate Mutex locking (No deadlocks, no race conditions).
4. **Fragmentation Analysis:** Verify that split_block is actually creating smaller blocks in lower size classes during execution.